Projeto de software

GameStore • Gestão de produtos, estoque e vendas Lucas Oliveira | 04 de novembro de 2026

1. Objetivo e enquadramento

Desenvolver um novo projeto Java SE no NetBeans com domínio reutilizável, conforme a alternativa permitida no enunciado. O projeto gerencia produtos e vendas de uma loja de jogos. O código desktop anterior não foi disponibilizado e não integra esta entrega. Não se atribui esta implementação a um projeto anterior não analisado.

Nesta etapa, a entrega é a preparação do código para reutilização, com regras separadas da interface. A interface atual é de console; o sistema web é uma evolução posterior. A aplicação não depende de Java Swing.

2. Usuário e fluxo principal

O operador cadastra produtos com nome, preço e estoque. Consulta a listagem para obter o identificador, registra uma venda e acompanha o estoque e o histórico. Também pode corrigir cadastros e excluir produtos. Todas as mudanças concluídas são persistidas em arquivo local.

3. Requisitos funcionais

ID	Requisito e critério de aceitação
RF01	Cadastrar produto e gerar um identificador UUID. Nome vazio ou duplicado, preço inválido e estoque negativo são rejeitados.
RF02	Listar produtos com identificador, nome, preço e estoque; indicar quando a lista está vazia.
RF03	Alterar nome, preço e estoque de um produto existente, preservando seu identificador.
RF04	Excluir produto existente sem apagar as vendas históricas.
RF05	Registrar uma venda com quantidade positiva e estoque suficiente, calcular seu total e atualizar o estoque.
RF06	Consultar histórico com nome e preço da época da venda, quantidade, data e total.
RF07	Salvar alterações e recuperar produtos e vendas ao reiniciar.

4. Regras de negócio

ID	Regra
RN01	Nomes são normalizados com trim; duplicidade ignora maiúsculas e minúsculas.
RN02	Preço deve ser maior que zero e conter no máximo duas casas decimais. Valores monetários usam BigDecimal.
RN03	Estoque é inteiro não negativo; quantidade vendida é inteira e estritamente positiva.
RN04	A venda não pode superar o estoque. Nenhuma mutação ocorre em caso de validação rejeitada.
RN05	O histórico guarda cópia do nome e preço e permanece após alteração ou exclusão do produto.
RN06	Produto, estoque e histórico são salvos como um único estado. Se a gravação falhar, o serviço conserva o estado anterior em memória.

5. Requisitos não funcionais

RNF01 — Compatibilidade: Java 17 ou superior, projeto Java SE/Ant reconhecível pelo NetBeans e sem dependências externas.

RNF02 — Manutenibilidade: entidades imutáveis, interface de repositório e separação entre domínio, serviço, armazenamento e apresentação.

RNF03 — Integridade: gravação em arquivo temporário e substituição atômica quando suportada pelo sistema de arquivos; em outros sistemas usa substituição convencional. Não há garantia contra queda de energia nem controle de concorrência entre processos.

RNF04 — Verificabilidade: testes executáveis por Main --test, resultados por cenário e falha explícita em caso de divergência.

6. Modelo de dados

Product: id, name, price, stock. Sale: id, productId, productName, quantity, unitPrice, date. StoreState: listas imutáveis de produtos e vendas. Uma venda faz referência lógica ao produto, mas preserva seu próprio retrato para permitir consulta após exclusão.

7. Limites e operação

Aplicação local para um operador e uma instância por arquivo. A pasta dados é criada na primeira gravação. O arquivo não é um banco de dados multiusuário. Não há login, pagamentos, emissão fiscal, estorno ou integração web nesta etapa. Faça backup de dados/gamestore.dat com a aplicação fechada.